Lookup, Don't Generate: A Capability Catalog as the Substrate for LLM Workflow Composition

Lookup, Don't Generate: A Capability Catalog as the Substrate for LLM Workflow Composition

> **Status: SEED (deferred paper #7).** The thesis — that a discoverable, effect/cost-annotated facet library turns "compose a workflow" from *code generation* into *lookup-then-compose*, and that each solved request enriches the substrate for the next (a "memory of solved requests" moat) — still needs a controlled composition experiment (§4). What exists today, and what this file captures, is a large **worked example** (§3) that instantiates every claim the paper will make. It is written up now, while the evidence is fresh, so the paper has a spine to grow around.

Abstract (stub)

LLM "workflow generation" usually means emitting bespoke code per task. We argue for the opposite: give the model a **capability catalog** — facets indexed by intent, annotated with effect (`pure` / `io` / `external`) and cost, matchable both at the facet level (`fw_capabilities`) and the whole-workflow level (`fw_catalog_match`) — and composition becomes *discovery plus wiring*. The integration surface is data (catalog rows), not handlers. We report a worked example in which a single national data pipeline over 99 heterogeneous information layers was composed almost entirely by catalog authorship and reuse of primitives from sibling domains, then run to completion over 3,167 fan-out units with zero failures. We close with the ablation the full claim still requires.

1. Introduction (outline)

The prevailing pattern — model reads a request, writes handler code — scales badly in exactly the way software always has: every new source is new code to write, test, and maintain, and nothing the model built yesterday makes today's request cheaper. We describe a substrate where it does. (Full treatment TODO; relate to Ray `runtime_env`, LangGraph/agent tool-catalogs, program synthesis by retrieval, and the reuse literature.)

2. The substrate (outline)

Three mechanisms, all shipped and documented elsewhere in this corpus, combine:

- **A facet capability index** (`fw_capabilities`) — search the deployed facet library *by intent* ("what operates on a GeoJSON path?"), returning signature, effect, and cost. The model discovers primitives instead of guessing names.
- **Effect/cost annotations** (`with Effect(...)`, `with Cost(...)`) — let the composer prefer pure/cheap primitives and see which steps hit an external engine, i.e. reason about a plan's shape before running it.
- **A reuse-first workflow catalog** (`fw_catalog_match`) — before authoring, match the request against past solutions; reuse or extend rather than re-derive. Each new workflow joins the catalog, so the corpus of solved requests monotonically grows.

The claim under test: with this substrate, the marginal cost of a new request falls as a function of how much of it is already *lookup*. §3 is one long instance of that falling cost.

3. Worked example: composing a 99-source national pipeline

3.1 The request

"A map per US county (~3,143), with a large, checkbox-toggleable taxonomy of information layers — boundaries, roads, parks, hydrology, healthcare and school points, demographics, health prevalence, environmental hazards, seismic and flood risk, climate normals, housing — plus a nation → state → county master index." The taxonomy alone named on the order of 250 candidate layers across 14 categories.

The *generate-a-handler-per-layer* reading of this task is hundreds of Python modules. That is the baseline we did **not** take.

3.2 What was actually composed

The pipeline (`fwh_county_atlas`) is, in the parts that matter, **not code**: it is a catalog — `layers.json`, 99 layers, 14 categories — plus one generic renderer that draws whatever a given county materialized, and a thin **source-field router** that dispatches each layer to a fetch adapter by the field it carries (`metric`, `census_var`, `places`, `epa_source`, `usgs_source`, `fema_source`, `noaa_source`, `hud_source`, `boundary_source`, `calc`). Adding a layer is adding a row.

The integration surface collapsed to **eight source-adapter modules covering seven fetch patterns** (OSM PBF tag-filter; ACS metric registry; Socrata county query; per-county REST + national-cache-then-bbox-clip; ArcGIS bbox envelope; per-state shapefile; derived/calc). 87 of the 99 layers wired; the 12 unwired are exactly the ones the current renderer *cannot* express (rasters needing tiles, a 500 MB national ZCTA set) — an honest, catalog-visible gap, not a silent omission.

3.3 The composition was mostly *reuse*, and it is countable

The adapters were not written from scratch. They are, in the majority, **discovered primitives from sibling domains**:

- **Direct code reuse:** the ACS choropleth path imports `census_us`'s metrics registry (`_lib.metrics`) verbatim; the climate layer imports `noaa_weather`'s station parser (`parse_stations`).
- **Vetted-endpoint reuse:** the EPA Superfund/Brownfields/TRI adapters reuse the exact EMEF/Envirofacts endpoints `save_earth` had already proven; the FEMA National Risk Index adapter reuses the ArcGIS FeatureServer `livability` already used.
- **Output reuse:** the entire tier-1 layer set is a tag-filter over the per-county PBF tree that `osm.planet` produces — the atlas *consumes* a fan-out another domain already ran.

So the substrate's claim is directly measurable here: **five sibling domains' primitives** (`osm.planet`, `census_us`, `noaa_weather`, `save_earth`, `livability`) were composed into a new domain, two by literal import and three by endpoint reuse, with per-source new code confined to the county-scoping glue (FIPS resolution, bbox clipping, the shared national cache). The request was answered far more by *lookup* than by *generation* — which is the whole thesis in one artifact.

3.4 The composed artifact ran at national scale

Composition that doesn't execute proves nothing. `BuildAtlasFanout(tier=3)` over the county PBF tree produced **3,167 county atlases, zero failed**, ~7.0 GB, then a 51-state master index — run by seven native runners on one host at ~23 s/county (the same campaign quantified in the informal-fleet report, §3). Two composition patterns were load-bearing at this scale and both are reusable catalog-level ideas, not atlas-specific hacks: **resolve-shared-geometry-once** (FIPS + tract geometry fetched once per county, shared across the census/health/FEMA adapters that all join to it) and **fetch-once national cache** (datasets with no county filter pulled once for the whole fan-out, bbox-filtered per county — without it, ~9,400 redundant source downloads).

3.5 The artifact also *enriched* the substrate

The moat claim is bidirectional. The atlas consumed five domains' primitives; it also **contributed** its own — the source-field router, the shared-national-cache pattern, the self-contained-SVG renderer with geometry dedup — each now a discoverable primitive the next request can find instead of reinvent. The catalog grew by exactly the solved request. This is the mechanism the paper claims makes request *N+1* cheaper than request *N*; the atlas is one measured step of it.

3.6 A second step: the moat compounds, at a different grade of reuse

A follow-on request — *map antibiotic-resistance gene spread across country, year, and species* — was answered by the `fwh_amr` domain, and it is the second measured point on the curve §4 asks for. Its reuse is real but of a **different grade** than §3.3, and the distinction is itself a finding:

- **Framework primitives, imported literally:** the backend-aware object-store layer (`facetwork.runtime.storage`, `facetwork.config`), the `DomainPackage` entry-point contract, and the name-filtered handler-registration protocol — the same surface the catalog exposes to every domain.
- **Sibling *templates*, copied and adapted (not imported):** the storage wrapper is `migration`'s, extended by two functions; the MapLibre year-slider world choropleth (basemap, play control, name search, provenance modal) is `migration`'s renderer restructured; the fan-out workflow (`ListOrganisms → foreach DownloadOrganism`) is the county atlas's `ListCounties → foreach BuildCountyAtlas` shape; the fetch-once shared-cache discipline is the county atlas's `_shared/` pattern. Crucially, the last two are **this corpus's own earlier solved requests** — and specifically two of the primitives §3.5 recorded the atlas *contributing back*. Request *N* (the atlas) measurably became substrate for request *N+1* (`amr`): the bidirectional loop closes on itself, once.
- **Cross-domain tooling:** the gallery publish reused `census_us`'s `publish_bundles` verbatim.

Genuinely **new code** was again confined to the source-specific core — the NCBI Pathogen Detection streaming ingest, the gene→drug-class classifier, and the `geo_loc_name`→ISO3 join — the same collapse the atlas showed, where new code shrinks to the adapter and the glue while scaffolding, storage, rendering, and orchestration are lookup. And it *ran*: 2.35 M isolates across 14 datasets → a world map + a 23,270-row gene × species table, in 85 s on one host.

The honest caveat is that most of this reuse is **template** reuse (copy-and-adapt) rather than the **literal-import** reuse of §3.3 — a weaker grade, because copied code is still maintained per-domain. But the *shape* replicated exactly across two independent requests: new code concentrated in the data adapter and the domain vocabulary; everything structural came from the substrate. Two points are not a curve — but they establish that the first was not a fluke, that reuse is **graded** (literal import ▸ template ▸ vetted-endpoint), and that the grade is what §4.3 must measure across a longer sequence.

4. What the full paper still needs (the honest gap)

This worked example *instantiates* the thesis; it does not yet *isolate* it. To be a research contribution rather than an experience report, the paper needs:

1. **A controlled comparison.** The same set of requests answered (a) generate-from- scratch and (b) lookup-then-compose over the catalog, measured on new-code volume, wall-clock-to-working, defect rate, and reuse count. The atlas gives the (b) datapoint; (a) is unrun.
2. **An effect/cost ablation.** We *assert* the annotations steered the composer toward pure/cheap primitives; we have not shown a composer *without* them making worse plans. Needed: composition with and without the mixins visible.
3. **A moat curve, not a moat anecdote.** §3.5–§3.6 give *two* steps — and §3.6 is the stronger kind, a request that reused a *prior request's* contributed primitives, so the N→N+1 loop is observed rather than only asserted. But two points are not a trend. The claim — marginal cost falling across a sequence of related requests — still needs a family of domains composed in order, plotting the lookup-vs-generation share (and its *grade*: literal-import vs template vs endpoint reuse) per step.
4. **Autonomy scoping, stated plainly.** Here "the LLM composed it" means Claude, in an agent loop, with a human gating design and reviewing every step. The paper must not overclaim autonomy; the interesting result holds even under supervision (the *substrate* did the reuse work), but the boundary must be explicit.

Until those exist, this file is the spine and the evidence, not the claim.